

РАЗРАБОТКА МНОГОПОТОЧНЫХ ПРИЛОЖЕНИЙ

Цель работы: получить практические навыки создания многопоточных приложений, используя классы библиотеки **FCL**.

Задачи работы:

- изучить технологию создания многопоточных приложений в среде разработки **Visual Studio** платформы **.NET**;
- изучить основные классы библиотеки **FCL**, предназначенные для создания многопоточных приложений.
- выполнить практическое задание по разработке приложения на языке **C#**.

Краткие теоретические сведения

С помощью языка **C#** можно создавать приложения, которые выполняют несколько задач одновременно, при этом задачи, которые могут потенциально задержать выполнение других задач, выполняются в отдельных потоках. Такой способ организации работы приложения называется *многопоточностью*, или *свободным созданием потоков*.

Поток (thread) можно представить как специфический путь выполнения внутри процесса **Win32**.

Многопоточность – это свойство платформы или приложения, состоящее в том, что процесс, порожденный в операционной системе, может состоять из нескольких потоков, выполняющихся параллельно, т. е. без предписанного порядка во времени. Такие потоки также называют *потоками выполнения* (от англ. thread of execution), иногда – *нитеями* (буквальный перевод слова thread) или *тредами*.

Процесс – это объект, который создается операционной системой для каждого приложения в момент его запуска. Процесс характеризуется собственным адресным пространством, которое недоступно другим процессам напрямую.

Сутью многопоточности является квазимногозадачность на уровне одного исполняемого процесса, т. е. выполнение всех потоков в адресном пространстве процесса. Кроме того, все потоки процесса имеют не только общее адресное пространство, но и общие дескрипторы файлов. Выполняющийся процесс имеет как минимум один – главный – поток.

Первый поток, создаваемый в процессе, называется *первичным* (primary thread). В любом процессе существует по крайней мере один

поток, который выполняет роль точки входа для приложения. В приложениях **.NET** такой точкой входа является метод **Main()**.

Многопоточные приложения создаются для достижения следующих целей:

- повышения производительности работы приложения;
- упрощения программы за счет использования общего адресного пространства;
- сокращения времени отклика приложения.

Примерами приложений, в которых целесообразно создавать дополнительные потоки, могут быть печать большого файла, длительный поиск в файле, выполнение большого объема вычислений, подключение к удаленному серверу.

Класс System.Threading.Thread. Первичный поток в приложении создается автоматически. Базовым классом для создания вторичных потоков является класс **Thread**, определенный в пространстве имен **System.Threading** и содержащий ряд методов для работы с потоками (табл. 9.1).

Работа с вторичными потоками в приложении состоит из следующих шагов:

- создания потока. Новый поток формируется путем объявления переменной типа **Thread** и вызова конструктора, которому предоставляется имя процедуры или метода, которые требуется выполнить в потоке. На этот метод накладываются некоторые ограничения: он не должен возвращать значение, отличное от **void**, и не должен принимать параметров;
- запуска потока на выполнение с помощью метода **Start**.

Таблица 9.1

Методы класса **Thread**

Метод	Результат
Start	Запускает поток
Sleep	Приостанавливает поток на определенное время
Suspend	Приостанавливает поток, когда он достигает безопасной точки
Abort	Останавливает поток, когда он достигает безопасной точки
Resume	Возобновляет работу приостановленного потока
Join	Приостанавливает текущий поток до тех пор, пока не будет завершен другой поток. При заданном времени ожидания этот метод возвращает значение True при условии окончания другого потока в течение этого времени

Потоку можно присвоить имя, используя свойство **Name**. Это создает большие удобства при отладке, так как имена потоков можно вывести или увидеть в окне **Debug–Threads** среды **Visual Studio**. Имя

поток может быть назначено в любой момент, но только один раз, поскольку при попытке его изменения будет сгенерировано исключение.

Главному потоку приложения также можно назначить имя.

Пример консольного приложения, в котором главному и вторичному потоку присвоены имена, представлен в листинге 9.1. В этом примере доступ к главному потоку осуществляется через статическое свойство **CurrentThread** класса **Thread**.

Листинг 9.1. Присваивание имен главному и вторичному потокам

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;

namespace поток1
{
    class Program
    {
        static void Main( )
        {
            // Присваивание имени main главному потоку,
            // создаваемому автоматически
            Thread.CurrentThread.Name = "main";

            // Создание объекта worker вторичного потока. Конструктору класса
            // Thread передается имя функции Go, которая будет выполняться в потоке
            Thread worker = new Thread(Go);

            // Присваивание имени worker вторичному потоку
            worker.Name = "worker";

            // Запуск вторичного потока на выполнение
            worker.Start();
            Go(); // Вызов функции для вывода имен запущенных потоков
            Console.ReadLine();
        }

        // Функция, которая выполняется в потоке
        static void Go( )
        {
            Console.WriteLine("Hello from " + Thread.CurrentThread.Name);
        }
    }
}
```

Результатом работы этой программы являются строки

Hello from main
Hello from worker

Основные и фоновые потоки. Выделяют два типа потоков: основные и фоновые.

Основные потоки характеризуются тем, что они не дают приложению завершиться, пока не завершатся сами. По умолчанию потоки создаются как основные, что означает, что приложение не будет завершено, пока один из таких потоков будет исполняться.

Пример разработки интерфейса приложения **Windows Forms**, имитирующего процесс форматирования диска, выполняемый в основном потоке, показан на рис. 9.1, а его код приведен в листинге 9.2. После запуска этого приложения мы сможем закрыть главное окно приложения во время форматирования, однако выполнение потока при этом не будет завершено, поэтому уже после закрытия окна приложения мы увидим сообщение о том, что форматирование завершено.

Фоновые потоки завершаются сразу после закрытия приложения.

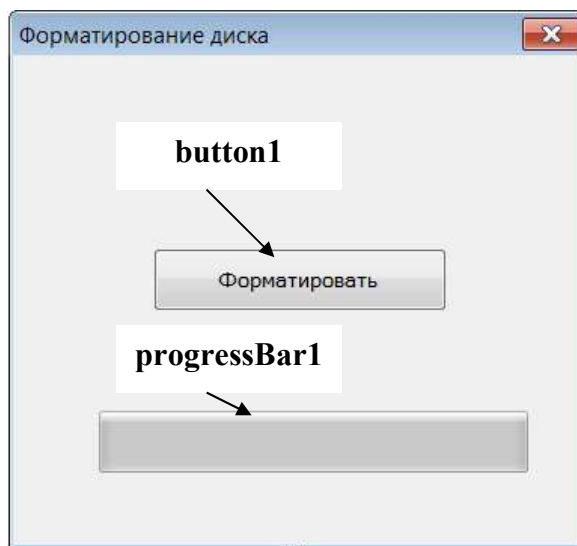


Рис. 9.1. Интерфейс приложения, имитирующего процесс форматирования диска

Листинг 9.2. Код формы приложения, имитирующего процесс форматирования диска (созданный поток является основным)

```
// Эта строка добавляется вручную  
using System.Threading;
```

```
public partial class Form1 : Form  
{  
    public Form1()
```

```

    {
        InitializeComponent();
    }

    // Функция-обработчик события щелчка по кнопке
    private void button1_Click(object sender, EventArgs e)
    {
        // Создание объекта потока, являющегося основным по умолчанию
        Thread myThread = new Thread(new Thread
            Start(formatMethod));

        // Запуск потока на выполнение
        myThread.Start();
    }
    // Метод, выполняемый во вторичном потоке
    private void formatMethod()
    {
        for (int i = 0; i < 100; i++)
        {

            // Приостановка выполнения вторичного потока на 100 мс
            Thread.Sleep(100);
            progressBar1.Increment(1);
        }

        MessageBox.Show("Форматирование завершено!");
    }
}

```

Каким быть потоку: основным или фазовым, определяет свойство **IsBackground**. Это свойство по умолчанию установлено в значение **false**, что означает, что поток основной. Если свойство **IsBackground** установить в значение **true**, то это будет говорить о том, что поток фоновый.

В листинге 9.3 показано, как следует изменить код метода-обработчика щелчка по кнопке из листинга 9.2, чтобы созданный поток стал фоновым. Теперь, если нажать кнопку закрытия окна, то приложение завершится вместе со своими потоками.

Листинг 9.3. Модификация кода функции-обработчика щелчка по кнопке в приложении, имитирующем процесс форматирования диска (созданный поток является фоновым)

```

    private void button1_Click(object sender, EventArgs e)
    {

        // Создание объекта потока с именем myThread
        Thread myThread = new Thread(new ThreadStart(formatMethod));
    }
}

```

```
// Установка потока как фонового
    myThread.IsBackground = true;

// Запуск потока на выполнение
    myThread.Start( );
}
```

Однако фоновые потоки имеют и определенные недостатки. Так, если приложение выполняет вычисления, то сделать этот поток фоновым вполне допустимо, но если поток выполняет только запись данных в файл или форматирует диск, то преждевременная остановка потока может привести к потере информации в файле или порче носителя. Поэтому подобные потоки необходимо делать основными.

Порядок выполнения работы

1. Изучить теоретические сведения и примеры, представленные выше.
2. Ответить на контрольные вопросы.
3. Разработать **Windows**-приложение в соответствии с вариантом практического задания.
4. Составить отчет в электронном виде, который должен содержать титульный лист, цель лабораторной работы, задание, ответы на контрольные вопросы, листинг программы и результаты ее работы.

Контрольные вопросы и задания

1. Дайте определение многопоточности потока.
2. Какой объект называется процессом?
3. Перечислите цели создания многопоточных приложений.
4. Какой поток называется первичным?
5. Приведите примеры приложений, в которых целесообразно создавать дополнительные потоки.
6. С помощью какого класса выполняется создание и управление потоками?
7. Каким образом потоку можно присвоить имя?
8. Какой параметр передается конструктору класса **Thread** при создании потока?
9. В чем состоят различия между основным и фоновым потоками?
10. Какое свойство класса **Thread** позволяет установить тип потока?

11. Каким образом можно приостановить выполнение потока на определенное время?

12. Перечислите проблемы, возникающие при создании многопоточных приложений.

Варианты практических заданий

Во всех вариантах заданий требуется создать приложение с интерфейсом **Windows Forms**.

Варианты заданий 1–16 приведены в табл. 9.2. В этих заданиях необходимо создать фоновый поток, в котором вычисление указанной в таблице функции выполняется с точностью $\xi = 0,000\ 01$, а сама функция представляется в виде суммы членов ряда. Кроме того, необходимо предусмотреть обработку введенных данных с проверкой их корректности и выдачу результата или сообщения об ошибке. Вычисленное значение следует выводить в компонент, отображающий информацию.

В вариантах заданий 17–22 требуется реализовать в фоновых потоках действия, указанные в условии задания.

Таблица 9.2

Варианты практических заданий 1–16

№ варианта	Функция	Формула для вычисления
1	e^x	$1 + \frac{x}{1!} + \frac{x^2}{2!} + \dots + \frac{x^n}{n!} + \dots$
2	$\sin x$	$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots + (-1)^{m-1} \frac{x^{2m-1}}{(2m-1)!} + \dots$

Продолжение табл. 9.2

№ варианта	Функция	Формула для вычисления
3	$\cos x$	$1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots + (-1)^m \frac{x^{2m}}{(2m)!} + \dots$
4	$\ln(1+x)$	$x - \frac{x^2}{2} + \frac{x^3}{3} - \dots + (-1)^{n-1} \frac{x^n}{n} + \dots, \quad x \in [-1; 1]$
5	$\ln(1-x)$	$-x - \frac{x^2}{2} - \frac{x^3}{3} - \dots + \frac{x^n}{n} - \dots, \quad x \in [-1; 1]$

6	$\ln \frac{1+x}{1-x}$	$2\left(x + \frac{x^3}{3} + \frac{x^5}{5} + \dots + \frac{x^{2m-1}}{2m-1} + \dots\right), \quad x \in [-1; 1]$
7	$\operatorname{sh} x$	$x + \frac{x^3}{3!} + \frac{x^5}{5!} + \dots + \frac{x^{2m-1}}{(2m-1)!} + \dots$
8	$\operatorname{ch} x$	$1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \dots + \frac{x^{2m}}{(2m)!} + \dots$
9	$\operatorname{arctg} x$	$x - \frac{x^3}{3} + \frac{x^5}{5} - \dots + (-1)^{m-1} \frac{x^{2m-1}}{2m-1} + \dots, \quad x \in [-1; 1]$
10	$(1+x)^\alpha$	$1 + \alpha x + \frac{\alpha(\alpha-1)}{2} x^2 + \dots + \frac{\alpha(\alpha-1)\dots(\alpha-n+1)}{n!} x^n + \dots,$ $x \in [-1; 1]$
11	$\frac{1}{1+x}$	$1 - x + x^2 - \dots + (-1)^n x^n + \dots, \quad x \in [-1; 1]$
12	$\frac{1}{1-x}$	$1 + x + x^2 + \dots + x^n + \dots, \quad x \in [-1; 1]$
13	$\frac{1}{(1-x)^2}$	$1 + 2x + 3x^2 + \dots + (n+1)x^n + \dots, \quad x \in [-1; 1]$
14	$\sqrt{1+x}$	$1 + \frac{x}{2} - \frac{1 \cdot x^2}{2^2 \cdot 2!} + \frac{1 \cdot 3 \cdot x^3}{2^3 \cdot 3!} - \frac{1 \cdot 3 \cdot 5 \cdot x^4}{2^4 \cdot 4!} + \dots +$ $+ (-1)^{n+1} \frac{1 \cdot 3 \cdot 5 \cdot (2n-3)x^n}{2^n n!}$

Окончание табл. 9.2

№ варианта	Функция	Формула для вычисления
15	$\operatorname{arth}(x)$	$x + \frac{x^3}{3} + \frac{x^5}{5} + \dots = \sum_{n=0}^{\infty} \frac{1}{2n+1} x^{2n+1}, \quad x < 1$
16	$\arcsin(x)$	$x + \frac{x^3}{6} + \frac{3x^5}{40} + \dots = \sum_{n=0}^{\infty} \frac{(2n)!}{4^n (n!)^2 (2n+1)} x^{2n+1}, \quad x < 1$

17. В фоновом потоке получить список простых чисел в указанном пользователем интервале.

18. Упорядочить содержимое файла по возрастанию, используя фоновый поток.

19. Создать в отдельном потоке **A** массив из **N** целых случайных чисел в диапазоне от –999 до 999 и вывести на экран эти числа.

Создание и вывод элементов массива должны производиться через заданное время **T**, значения **N** и **T** – вводиться пользователем до запуска процесса. Массив должен обрабатываться двумя другими потоками **B** и **C**, работающими параллельно с потоком, создающим массив. Все потоки должны выводить результаты своей работы в текстовые окна, каждый поток в свое окно.

20. Создать четыре потока: **main** (главный поток), который запускает потоки **inc**, **dec** и **print**.

Главный поток должен постоянно (каждые 10 мс) проверять значение переменной **ACCOUNT** и завершать процесс, если эта переменная вышла за границы диапазона $[-10000, +10000]$.

Поток **inc** должен время от времени (паузы выбираются случайным образом от 100 мс и до 3 с включительно) увеличивать значение переменной **ACCOUNT** на некоторую случайную величину, например от 1 до 100.

Поток **dec** должен время от времени (паузы выбираются случайным образом от 100 мс и до 3 с включительно) уменьшать значение переменной **ACCOUNT** на некоторую случайную величину, например от 1 до 100.

Поток **print** должен выводить на экран новое значение переменной **ACCOUNT**, как только оно изменилось.

21. Создать два потока: главный и вторичный. Главный поток программы должен считывать вводимые пользователем строки и помещать их в начало связанного списка. При вводе пустой строки программа должна выдавать текущее состояние списка. Вторичный поток должен пробуждаться каждые 5 с и сортировать список в алфавитном порядке.

22. В фоновом потоке выполнить вычисление числа π по ряду Лейбница с точностью до четвертого знака после запятой. Ряд Лейбница

имеет вид $\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots \right)$.